

Nexus

Diseño e Implementación de un Lenguaje

Autómatas, Teoría de Lenguajes y Compiladores
Ingeniería Informática

Índice

1. Grupo	3
2. Repositorio	3
3. Dominio	3
4. Construcciones	4
4.1. Sintaxis general de un programa	4
4.1.1. Significado de cada sección	4
4.2. Declaración de grafos	5
4.3. Declaración de nodos	6
4.3.1. Semántica de los atributos de nodo	6
4.3.2. Compatibilidades entre atributos	7
4.4. Declaración de aristas	7
4.5. Reglas estructurales	7
4.6. Grupos de nodos	7
4.7. Restricciones	8
4.8. Grafos derivados	8
4.9. Bloques de análisis	9
4.10. Algoritmos base	9
4.11. Exportación	9
4.12. Comentarios	10
4.13. Futuras extensiones	10
5. Validaciones semánticas	10
5.1. Validaciones sobre nodos y atributos semánticos	10

5.2.	Compatibilidad entre atributos de arista y <i>traits</i>	11
5.3.	Compatibilidad entre operador de arista y tipo de grafo	11
5.4.	Compatibilidad entre restricciones y tipo de grafo	11
5.5.	Compatibilidad entre transformaciones y tipo de grafo	11
5.6.	Compatibilidad entre algoritmos y tipo de grafo	12
5.7.	Validaciones de referencias y consistencia interna	12
6.	Casos de prueba	12
6.1.	Casos de prueba de aceptación	12
6.2.	Casos de prueba de rechazo	13
7.	Ejemplos	14
7.1.	Ejemplo 1: grafo dirigido y ponderado con grupos, restricciones, grafos derivados y análisis	14
7.2.	Ejemplo 2: grafo dirigido que satisface restricciones de árbol y árbol binario	15
7.3.	Ejemplo 3: red de flujo dirigida y capacitada	16

1. Grupo

Nombre	Apellido	Legajo	E-mail
Pablo	Gorostiaga	65148	pgorostiaga@itba.edu.ar
Franco	Pampuri	65552	fpampuri@itba.edu.ar
Jonas	Glaubart	65790	jglaubart@itba.edu.ar
Eugenio	Migliaro	65508	emigliaro@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en el siguiente [repositorio](#).

3. Dominio

El objetivo del proyecto es desarrollar *Nexus*, un lenguaje específico de dominio orientado a la **definición, validación, transformación y análisis de grafos**. El lenguaje permitirá describir grafos como objetos matemáticos de manera declarativa, indicando su estructura, sus propiedades y los análisis que se desean ejecutar sobre ellos.

Nexus permitirá modelar:

- grafos dirigidos y no dirigidos;
- grafos ponderados;
- grafos capacitados.

Un programa en *Nexus* podrá:

- declarar nodos y aristas;
- definir grupos de nodos;
- establecer restricciones estructurales;
- derivar nuevos grafos a partir de transformaciones predefinidas;
- ejecutar algoritmos clásicos de teoría de grafos;
- exportar grafos y resultados en formato DOT y JSON.

En esta primera etapa no se incluirán:

- generadores paramétricos;
- construcciones imperativas;
- mecanismos generales de programación.

El foco del proyecto estará puesto en una especificación clara, consistente y formalmente verificable, priorizando un núcleo reducido pero expresivo del lenguaje.

4. Construcciones

4.1. Sintaxis general de un programa

Un programa en *Nexus* podrá estar compuesto por una o más declaraciones de grafos. Cada declaración de grafo podrá incluir, además de su tipo, bloques para nodos, aristas, grupos y restricciones.

Luego de declarar un grafo, el lenguaje permitirá:

- derivar nuevos grafos a partir de transformaciones predefinidas;
- definir bloques de análisis sobre un grafo ya declarado;
- ejecutar algoritmos dentro de esos bloques de análisis;
- exportar tanto el grafo como los resultados producidos.

La forma general de un programa será la siguiente:

```
graph <graph_id>:  
  kind directed  
  traits weighted capacitated  
  
  nodes:  
    <declaraciones de nodos>  
  
  edges:  
    <declaraciones de aristas>  
  
  groups:  
    <declaraciones de grupos>  
  
  constraints:  
    <declaraciones de restricciones>  
  
derive <new_graph_id> from <graph_id> using <transformation>  
  
analysis <analysis_id> on <graph_id>:  
  <sentencias de analisis>
```

Los bloques *traits*, *groups* y *constraints* son opcionales. Sin embargo, cuando aparezcan, deberán respetar exactamente la sintaxis del lenguaje y ubicarse dentro de la declaración del grafo correspondiente.

4.1.1. Significado de cada sección

- `graph <graph_id>`: inicia la declaración de un nuevo grafo y fija el identificador con el que podrá ser referenciado posteriormente.

- `kind directed` o `kind undirected` indica si el grafo es dirigido o no dirigido. Esta línea es obligatoria.
- `traits` permite declarar propiedades adicionales del grafo. En esta versión, los *traits* disponibles son `weighted` y `capacitated`.
- `nodes`: introduce el bloque donde se declaran los nodos del grafo.
- `edges`: introduce el bloque donde se declaran las aristas del grafo.
- `groups`: introduce un bloque opcional donde se definen subconjuntos nombrados de nodos.
- `constraints`: introduce un bloque opcional donde se expresan propiedades que el grafo debe satisfacer.
- `derive <new_graph_id> from <graph_id> using <transformation>` permite construir un nuevo grafo a partir de uno ya existente.
- `analysis <analysis_id> on <graph_id>`: define un bloque de análisis asociado a un grafo declarado previamente.

4.2. Declaración de grafos

Los grafos se declararán mediante la construcción principal `graph <id>::`. Dentro de esa declaración, el usuario deberá indicar obligatoriamente el tipo del grafo mediante `kind`.

Ejemplo mínimo de un grafo dirigido:

```
graph G:
  kind directed

  nodes:
    a
    b

  edges:
    a -> b
```

Ejemplo de un grafo no dirigido, ponderado y capacitado:

```
graph H:
  kind undirected
  traits weighted capacitated

  nodes:
    x
    y

  edges:
    x -- y weight 4 capacity 8
```

En esta primera versión del lenguaje, los *traits* admitidos son:

- **weighted**, que habilita el uso del atributo **weight** en aristas;
- **capacitated**, que habilita el uso del atributo **capacity** en aristas.

Un mismo grafo podrá declarar ninguno, uno o ambos *traits*.

4.3. Declaración de nodos

El lenguaje permitirá declarar nodos dentro de un bloque **nodes**. Cada nodo deberá tener un identificador único dentro del grafo y podrá incluir atributos semánticos como **root**, **source**, **sink** o **terminal**. Estos atributos no deben interpretarse como meras etiquetas descriptivas, sino como anotaciones con semántica verificable por el compilador.

Ejemplo de bloque de nodos:

```
nodes:
  r root
  s source
  t sink
  a
  b terminal
```

4.3.1. Semántica de los atributos de nodo

root Designa el vértice raíz del grafo. Puede existir a lo sumo un nodo de este tipo por grafo. Si el grafo es dirigido, el nodo marcado como **root** debe cumplir $\text{indegree}(\text{root}) = 0$. Toda restricción de `tree rooted_at r` o `binary_tree rooted_at r` deberá referirse al único nodo marcado como **root**.

source Designa el nodo de origen principal de un grafo dirigido. Solo puede declararse en grafos **directed**. Puede existir a lo sumo un nodo **source** por grafo. Debe cumplir $\text{indegree}(\text{source}) = 0$ y $\text{outdegree}(\text{source}) > 0$.

sink Designa el nodo de destino principal de un grafo dirigido. Solo puede declararse en grafos **directed**. Puede existir a lo sumo un nodo **sink** por grafo. Debe cumplir $\text{outdegree}(\text{sink}) = 0$ e $\text{indegree}(\text{sink}) > 0$.

terminal Designa un nodo terminal u hoja. En grafos dirigidos, un nodo **terminal** debe cumplir $\text{outdegree}(\text{terminal}) = 0$. En grafos no dirigidos, un nodo **terminal** debe cumplir $\text{degree}(\text{terminal}) = 1$. Puede haber múltiples nodos **terminal** dentro del mismo grafo.

4.3.2. Compatibilidades entre atributos

- Un mismo nodo no podrá ser simultáneamente **source** y **sink**.
- En grafos dirigidos, todo nodo **sink** será terminal en términos estructurales, aunque el atributo **terminal** no aparezca explícitamente.
- Un nodo marcado como **root** no podrá ser **sink** ni **terminal**, salvo en el caso degenerado de un grafo de un único nodo y sin aristas.

4.4. Declaración de aristas

El lenguaje permitirá declarar aristas dentro de un bloque **edges**. La sintaxis concreta dependerá del tipo de grafo.

En grafos dirigidos se utilizará el operador **->**:

```
edges:
  a -> b
  b -> c weight 5
  c -> d weight 2 capacity 7
```

En grafos no dirigidos se utilizará el operador **--**:

```
edges:
  x -- y
  y -- z weight 4
```

Las aristas podrán incluir atributos como **weight** y **capacity**, siempre que el grafo haya sido declarado con los *traits* correspondientes.

4.5. Reglas estructurales

El lenguaje permitirá **self-loops**. En cambio, no permitirá aristas paralelas. Es decir, entre dos vértices podrá existir como máximo una arista, considerando pares ordenados en grafos dirigidos y pares no ordenados en grafos no dirigidos.

No obstante, si un grafo incluye la restricción **assert acyclic**, el compilador deberá rechazar toda arista de la forma **u ->u** o **u - u**, ya que todo *self-loop* constituye un ciclo.

4.6. Grupos de nodos

El lenguaje permitirá declarar subconjuntos nombrados de nodos mediante un bloque **groups**. Estos grupos podrán utilizarse para expresar restricciones y para construir grafos derivados.

Ejemplo:

```
groups:
  core = {lexer, parser, ast}
```

```
leaves = {c, d, e}
```

Cada grupo deberá tener nombre único dentro del grafo y solo podrá contener nodos previamente declarados en el bloque `nodes`.

4.7. Restricciones

El lenguaje incluirá un bloque `constraints` para definir propiedades que deben cumplirse sobre el grafo. Dentro de este bloque, cada restricción deberá estar precedida por la palabra clave `assert`.

Entre las restricciones previstas se incluyen:

- `assert connected`
- `assert strongly_connected`
- `assert acyclic`
- `assert reachable a ->b`
- `assert tree rooted_at r`
- `assert binary_tree rooted_at r`
- `assert forall n in <group>: <predicado>`

Para las restricciones cuantificadas, el lenguaje permitirá expresiones numéricas simples basadas en:

- `indegree(n)`
- `outdegree(n)`
- `degree(n)`

En esta primera versión se proponen comparaciones claras y acotadas, tales como: `=`, `!=`, `>=`, `<=`, `<` y `>`.

4.8. Grafos derivados

El lenguaje permitirá crear nuevos grafos a partir de transformaciones predefinidas mediante la construcción:

```
derive <new_graph_id> from <graph_id> using <transformation>
```

Las transformaciones iniciales propuestas son:

- `transpose`

- `induced_subgraph(<group>)`
- `remove_self_loops`
- `underlying`

Ejemplos:

```
derive ReverseBuild from Build using transpose
```

```
derive CoreBuild from Build using induced_subgraph(core)
```

```
derive SimpleFlow from Flow using remove_self_loops
```

4.9. Bloques de análisis

El lenguaje permitirá definir bloques `analysis <id>on <grafo>` para ejecutar algoritmos sobre un grafo. Dentro de cada bloque, los resultados deberán poder nombrarse con `as <id>`.

Ejemplo:

```
analysis Main on Build:
  run shortest_path from lexer to emit as critical_path
  export graph to dot
  export result critical_path to json
```

4.10. Algoritmos base

La versión inicial de *Nexus* contempla el siguiente conjunto de algoritmos:

- `shortest_path` from `<u>` to `<v>`
- `topological_sort`
- `components`
- `scc` (*Strongly Connected Components*)
- `mst` (*Minimum Spanning Tree*)
- `max_flow` from `<s>` to `<t>`

4.11. Exportación

El lenguaje permitirá exportar grafos y resultados de análisis en:

- DOT

- JSON

Las formas válidas serán:

```
export graph to dot
export graph to json
export result <id> to dot
export result <id> to json
```

4.12. Comentarios

Se permitirá el uso de comentarios de línea con `//`.

Ejemplo:

```
// Este es un comentario valido
```

4.13. Futuras extensiones

Como extensión posterior, se propone incorporar un subsistema de `generator/build` para la construcción paramétrica de grafos. La idea no es exponer constructores de grafos famosos, sino permitir que el usuario defina sus propios generadores con parámetros, variables locales, condicionales y estructuras iterativas acotadas, para luego instanciarlos dentro de un grafo. Esta extensión se deja fuera del alcance principal de esta entrega para no comprometer la viabilidad de las etapas posteriores.

5. Validaciones semánticas

El compilador deberá validar la consistencia entre la declaración del grafo, sus atributos, sus restricciones, sus transformaciones y sus análisis.

5.1. Validaciones sobre nodos y atributos semánticos

- No deberán existir dos nodos marcados como `root` dentro del mismo grafo.
- No deberán existir dos nodos marcados como `source` dentro del mismo grafo.
- No deberán existir dos nodos marcados como `sink` dentro del mismo grafo.
- Los atributos `source` y `sink` solo podrán aparecer en grafos dirigidos.
- Todo nodo marcado como `terminal` deberá satisfacer la condición estructural correspondiente al tipo de grafo.
- Todo nodo marcado como `source` deberá satisfacer `indegree = 0`.
- Todo nodo marcado como `sink` deberá satisfacer `outdegree = 0`.

- Toda restricción de árbol enraizado deberá usar como raíz al único nodo marcado como `root`.
- Si un análisis ejecuta `max_flow from <s> to <t>` sobre un grafo que declara `source` y `sink`, entonces `<s>` y `<t>` deberán coincidir con dichos nodos.

5.2. Compatibilidad entre atributos de arista y *traits*

- El atributo `weight` solo podrá aparecer en aristas de grafos declarados con el *trait* `weighted`.
- El atributo `capacity` solo podrá aparecer en aristas de grafos declarados con el *trait* `capacitated`.
- Si un grafo no fue declarado como `weighted`, toda aparición de `weight` deberá rechazarse.
- Si un grafo no fue declarado como `capacitated`, toda aparición de `capacity` deberá rechazarse.

5.3. Compatibilidad entre operador de arista y tipo de grafo

- En grafos `directed` solo se permitirá el operador `->`.
- En grafos `undirected` solo se permitirá el operador `-`.
- El compilador deberá rechazar cualquier arista cuyo operador no coincida con el `kind` del grafo.

5.4. Compatibilidad entre restricciones y tipo de grafo

- `assert connected` solo deberá ser válido sobre grafos no dirigidos.
- `assert strongly_connected` solo deberá ser válido sobre grafos dirigidos.
- `assert tree rooted_at <r>` solo deberá ser válido sobre grafos dirigidos.
- `assert binary_tree rooted_at <r>` solo deberá ser válido sobre grafos dirigidos.
- `assert reachable <u> -> <v>` requerirá que ambos nodos existan previamente.
- Si un grafo incluye `assert acyclic`, el compilador deberá rechazar toda aparición de ciclos, incluyendo *self-loops*.

5.5. Compatibilidad entre transformaciones y tipo de grafo

- `transpose` solo deberá ser válido sobre grafos dirigidos.
- `underlying` solo deberá ser válido sobre grafos dirigidos, y producirá un grafo no dirigido.

- `induced_subgraph(<group>)` solo deberá ser válido si `<group>` fue declarado previamente en el grafo origen.
- `remove_self_loops` podrá aplicarse a cualquier grafo.

5.6. Compatibilidad entre algoritmos y tipo de grafo

- `topological_sort` solo deberá ser válido sobre grafos dirigidos y acíclicos.
- `components` solo deberá ser válido sobre grafos no dirigidos.
- `scc` solo deberá ser válido sobre grafos dirigidos.
- `mst` solo deberá ser válido sobre grafos no dirigidos y ponderados.
- `max_flow` solo deberá ser válido sobre grafos dirigidos y capacitados.
- `shortest_path` requerirá la existencia previa de los nodos origen y destino.

5.7. Validaciones de referencias y consistencia interna

- Todo nodo referido en una arista, restricción o algoritmo deberá haber sido declarado previamente.
- Todo grupo utilizado en una restricción cuantificada o en `induced_subgraph(<group>)` deberá haber sido declarado previamente.
- Todo grafo usado en `derive` o `analysis` deberá haber sido declarado previamente.
- Todo identificador de resultado exportado deberá haber sido producido previamente por una sentencia `run` dentro del mismo bloque `analysis`.
- No se permitirán grupos vacíos si luego son usados en restricciones cuantificadas que exijan propiedades estructurales de sus elementos.

6. Casos de prueba

Se proponen los siguientes casos iniciales de prueba.

6.1. Casos de prueba de aceptación

1. Se debe poder declarar un grafo dirigido con un único nodo `root` y múltiples nodos `terminal`.
2. Se debe poder declarar un grafo dirigido y capacitado con un único `source` y un único `sink`, y ejecutar `max_flow` entre ellos.
3. Se debe poder declarar un grafo dirigido y acíclico y ejecutar `topological_sort`.
4. Se debe poder declarar un grafo no dirigido y ponderado y ejecutar `mst`.

5. Se debe poder declarar un grafo dirigido y ponderado y ejecutar `shortest_path`.
6. Se debe poder declarar un grafo no dirigido y ejecutar `components`.
7. Se debe poder declarar un grafo dirigido con ciclos y ejecutar `scc`.
8. Se debe poder declarar ciclos cuando el grafo no tenga una restricción de aciclicidad.
9. Se debe poder definir un `group` y utilizarlo dentro de una restricción cuantificada.
10. Se debe poder derivar un subgrafo inducido a partir de un `group` y luego analizarlo.
11. Se debe poder derivar el `transpose` de un grafo dirigido y luego ejecutar `reachable` o `shortest_path` sobre el grafo derivado.
12. Se debe poder aplicar `remove_self_loops` a un grafo que incluya bucles, obteniendo un nuevo grafo sin ellos.
13. Se debe poder exportar grafos y resultados a DOT y JSON.
14. Se debe poder declarar un grafo con `source` y `sink` explícitos y usar exactamente esos nodos en un análisis de `max_flow`.
15. Se debe poder declarar un árbol dirigido con `assert tree rooted_at r` y restricciones cuantificadas sobre el grupo de hojas.
16. Se debe poder declarar un *binary tree* dirigido con `assert binary_tree rooted_at r` y verificar que todo nodo interno tenga `outdegree <= 2`.

6.2. Casos de prueba de rechazo

1. No se debe permitir declarar dos nodos con el mismo identificador dentro del mismo grafo.
2. No se debe permitir declarar dos nodos `root` en un mismo grafo.
3. No se debe permitir declarar dos nodos `source` en un mismo grafo.
4. No se debe permitir declarar dos nodos `sink` en un mismo grafo.
5. No se debe permitir declarar un nodo `source` en un grafo no dirigido.
6. No se debe permitir declarar un nodo `sink` en un grafo no dirigido.
7. No se debe permitir declarar un nodo `terminal` que tenga salidas en un grafo dirigido.
8. No se debe permitir declarar una arista cuyos extremos no hayan sido declarados previamente.
9. No se debe permitir usar `capacity` en un grafo que no haya sido declarado como `capacitated`.
10. No se debe permitir usar el operador `->` en un grafo no dirigido.

11. No se debe permitir usar el operador `-` en un grafo dirigido.
12. No se debe permitir ejecutar `topological_sort` sobre un grafo no dirigido o cíclico.
13. No se debe permitir ejecutar `components` sobre un grafo dirigido.
14. No se debe permitir ejecutar `mst` sobre un grafo no ponderado.
15. No se debe permitir ejecutar `max_flow` sobre un grafo no dirigido.
16. No se debe permitir ejecutar `max_flow` sobre un grafo no capacitado.
17. No se debe permitir aplicar `transpose` sobre un grafo no dirigido.
18. No se debe permitir aplicar `underlying` sobre un grafo no dirigido.
19. No se debe permitir derivar `induced_subgraph(<group>)` si `<group>` no fue declarado previamente.
20. No se debe permitir usar un grupo no declarado dentro de una restricción `forall`.
21. No se debe permitir declarar `assert connected` sobre un grafo dirigido.
22. No se debe permitir declarar `assert strongly_connected` sobre un grafo no dirigido.
23. No se debe permitir declarar `assert tree rooted_at <r>` sobre un grafo no dirigido.
24. No se debe permitir usar `weight` en un grafo que no haya sido declarado como `weighted`.
25. No se debe permitir ejecutar `scc` sobre un grafo no dirigido.
26. No se debe permitir ejecutar `mst` sobre un grafo dirigido.

7. Ejemplos

7.1. Ejemplo 1: grafo dirigido y ponderado con grupos, restricciones, grafos derivados y análisis

Este ejemplo muestra un grafo dirigido y ponderado, con grupos, restricciones de aciclicidad y alcanzabilidad, dos grafos derivados y dos bloques de análisis.

```
graph Build:
  kind directed
  traits weighted

  nodes:
    lexer
    parser
    ast
```

```

    emit

edges:
  lexer -> parser weight 2
  parser -> ast weight 3
  ast -> emit weight 4

groups:
  core = {lexer, parser, ast}
  sinks = {emit}

constraints:
  assert acyclic
  assert reachable lexer -> emit
  assert forall n in sinks: outdegree(n) = 0

derive CoreBuild from Build using induced_subgraph(core)
derive ReverseBuild from Build using transpose

analysis Main on Build:
  run shortest_path from lexer to emit as critical_path
  export graph to dot
  export result critical_path to json

analysis Reduced on CoreBuild:
  run shortest_path from lexer to ast as core_path
  export graph to json
  export result core_path to json

```

7.2. Ejemplo 2: grafo dirigido que satisface restricciones de árbol y árbol binario

Este ejemplo muestra un grafo dirigido que debe satisfacer propiedades de árbol enraizado y *binary tree*.

```

graph T:
  kind directed

  nodes:
    r root
    a
    b
    c terminal
    d terminal

  edges:
    r -> a
    r -> b
    a -> c

```

```

a -> d

groups:
  leaves = {b, c, d}
  internal = {r, a}

constraints:
  assert tree rooted_at r
  assert binary_tree rooted_at r
  assert forall n in leaves: outdegree(n) = 0
  assert forall n in internal: outdegree(n) <= 2

analysis TreeCheck on T:
  run shortest_path from r to d as path_rd
  export graph to dot
  export result path_rd to json

```

7.3. Ejemplo 3: red de flujo dirigida y capacitada

Este ejemplo muestra una red de flujo con source y sink explícitos, capacidades sobre aristas y un análisis de flujo máximo.

```

graph Flow:
  kind directed
  traits capacitated

  nodes:
    s source
    a
    b
    t sink

  edges:
    s -> a capacity 10
    s -> b capacity 5
    a -> b capacity 4
    a -> t capacity 8
    b -> t capacity 7

  groups:
    middle = {a, b}

  constraints:
    assert reachable s -> t
    assert forall n in middle: indegree(n) >= 1

analysis MaxFlowCheck on Flow:
  run max_flow from s to t as flow_st
  export graph to dot
  export result flow_st to json

```
